SAMSUN ÜNİVERSİTESİ

Mühendislik ve Doğa Bilimleri Fakültesi

Yazılım Mühendisliği

YZM102 / MYAZ104 PROGRAMLAMAYA GİRİŞ II DÖNEM SONU PROJE ÖDEVİ

SANAYİ YEDEK PARÇA YÖNETİM SİSTEMİ

Hazırlayan: Baran Özbey

Öğrenci No: 241478079

Dersi Veren Öğretim Elemanı:Arş.Gör. İLKER GÜR

Teslim Tarihi: 13 Mayıs 2026

İÇİNDEKİLER

1. GİRİŞ ... 3
2. SİSTEM TASARIMI VE VERİTABANI MİMARİSİ ... 4
3. KULLANILAN PYTHON KONULARI ... 6
4. UYGULAMA ARAYÜZÜ VE EKRAN GÖRÜNTÜLERİ ... 8
5. KARŞILAŞILAN ZORLUKLAR VE ÇÖZÜM YAKLAŞIMLARI ... 12
6. SONUÇ ... 13
7. KAYNAKÇA ... 14

1. GİRİŞ

Günümüz endüstriyel üretim ve sanayi sektörlerinde, operasyonların aksamadan yürütülebilmesi için yedek parça ve stok takibinin anlık olarak yapılması kritik bir öneme sahiptir. Manuel olarak veya basit elektronik tablolar (Excel vb.) üzerinden yürütülen stok yönetim süreçleri; insan hatasına açıklığı, veri kaybı riski ve anlık analiz yetersizlikleri gibi nedenlerle işletmelere zaman ve maliyet kayıpları yaşatmaktadır.

Bu rapor kapsamında detaylandırılan "Sanayi Yedek Parça Yönetim Sistemi" projesi, tam olarak bu endüstriyel ihtiyaca modern bir yazılım çözümü sunmak amacıyla Programlamaya Giriş II dersi bitirme ödevi olarak geliştirilmiştir. Geliştirilen bu yazılım, depo yöneticilerinin binlerce yedek parçayı kategorize edebilmesini, tedarikçi ilişkilerini yönetebilmesini ve anlık stok giriş-çıkış (log) hareketlerini takip edebilmesini sağlayan dinamik bir otomasyon sistemidir.

Projenin temel motivasyonu, ders kapsamında teorik olarak işlenen Python programlama dili konseptlerinin, veri tabanı yönetiminin ve ön yüz (front-end) teknolojilerinin gerçek ve komplike bir senaryoda birleştirilmesidir. Sistem mimarisi oluşturulurken MVC (Model-View-Controller) benzeri bir yaklaşım benimsenmiş; arka uçta (back-end) Python tabanlı Flask mikro çatısı, veri depolama birimi olarak hafif ve güvenilir olan SQLite3 tercih edilmiştir. Geleneksel stok takip uygulamalarından farklı olarak projeye veri bilimi araçları olan Pandas, Numpy ve Matplotlib kütüphaneleri de entegre edilmiş, böylece depodaki ürünlerin finansal değerleri ve hareket trendleri görsel grafiklere dönüştürülerek yönetime hazır raporlar sunma yeteneği kazandırılmıştır.

2. SİSTEM TASARIMI VE VERİTABANI MİMARİSİ

Projede, ilişkisel veritabanı modeli benimsenmiş ve verilerin güvenli, tutarlı bir şekilde saklanması için SQLite3 kullanılmıştır. Sistemde birbiriyle ilişkili toplam 4 adet tablo bulunmaktadır. Veri bütünlüğünü sağlamak amacıyla tablolarda PRIMARY KEY, FOREIGN KEY, UNIQUE ve NOT NULL gibi kısıtlamalar aktif olarak kullanılmıştır.

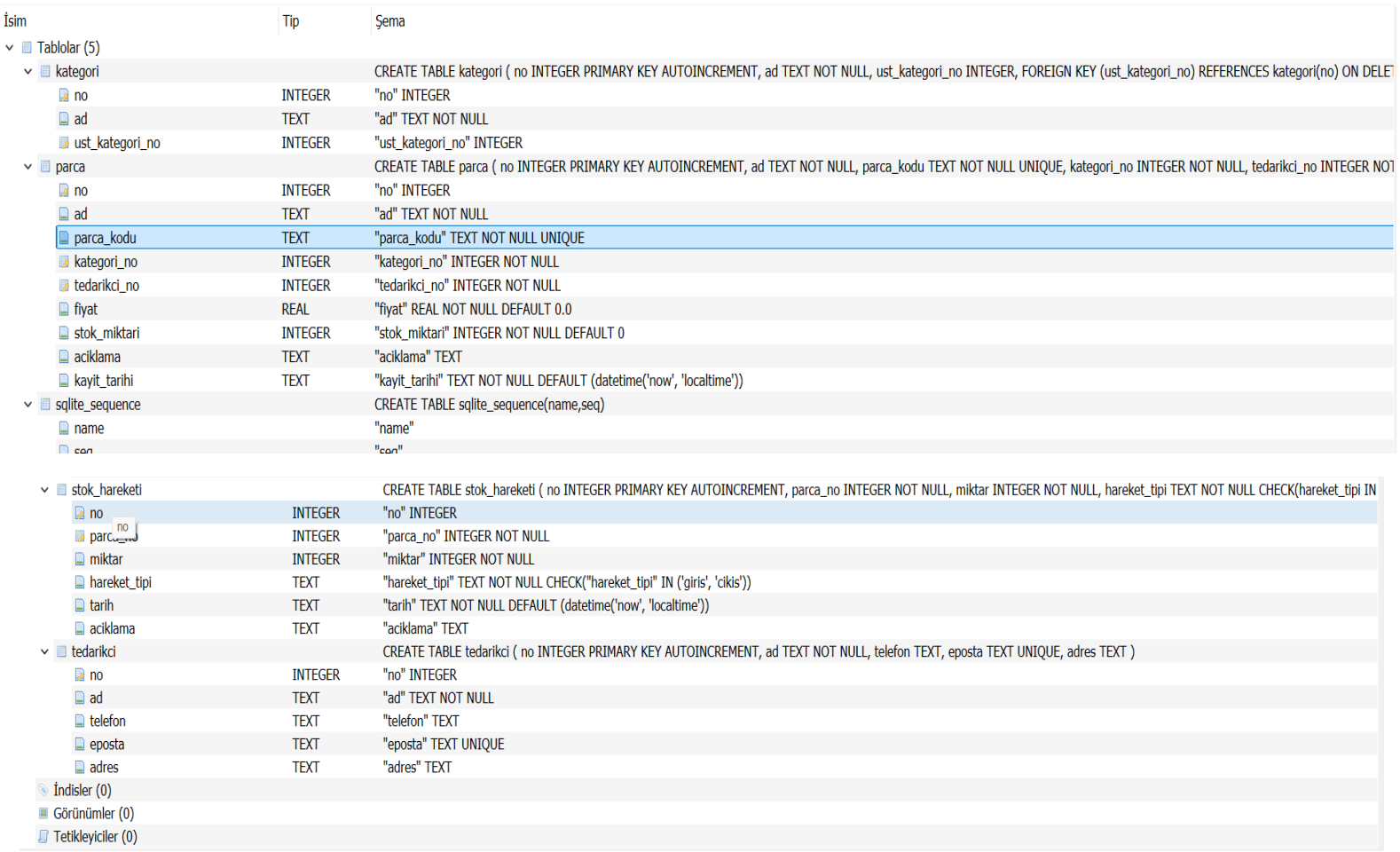
Veritabanını oluşturan tablolar ve işlevleri şunlardır:

1. Kategori Tablosu: Yedek parçaların hiyerarşik bir düzende tutulduğu tablodur. Kendi içinde özyinelemeli (recursive) bir yapıya sahiptir; yani bir kategorinin üst kategorisi yine aynı tablodan referans alınır (Self-Referencing Foreign Key).

2. Tedarikçi Tablosu: Parçaları temin eden firmaların iletişim ve adres bilgilerinin tutulduğu tablodur.

3. Parça Tablosu: Sistemin ana tablosudur. Her parçanın kendine ait benzersiz bir "parça_kodu" bulunmaktadır. Bu tablo, kategori ve tedarikçi tablolarıyla ilişkili (Foreign Key) çalışmaktadır.

4. Stok Hareketi Tablosu: Parçalara ait giriş ve çıkış işlemlerinin kaydedildiği log tablosudur. Bu sayede her parçanın geçmiş hareketi takip edilebilmekte ve güncel stok hesaplanabilmektedir.



Şekil 1: Veritabanı ER Diyagramı

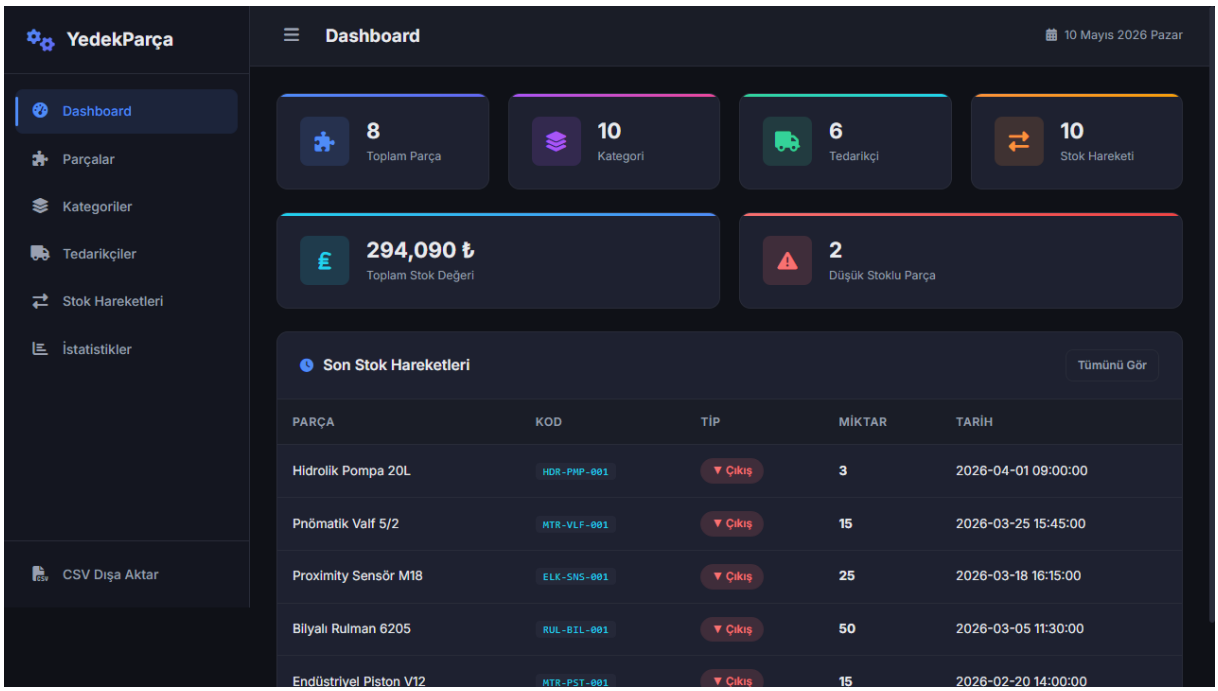
3. KULLANILAN PYTHON KONULARI

1. Değişkenler ve Veri Tipleri: SQLite veri tabanından alınan veriler (örneğin stok miktarının int, fiyatın float, isimlerin string olarak gelmesi) Python içinde doğru veri tiplerine dönüştürülerek matematiksel işlemlerde ve string formatlamalarında kullanılmıştır.
2. Koşul İfadeleri ve Döngüler (If/Elif/Else & For/While): Uygulamanın bel kemiğini oluşturur. Kullanıcıdan gelen HTML form verilerinin doğrulanmasında (boş kayıt engelleme), stoktan parça çıkışı yapılırken "eksi stoka düşmeme" kontrollerinde koşul ifadeleri kullanılmıştır. Çoklu veri listelerinin HTML tablolarına dönüştürülmesi sürecinde (Jinja2 template içerisinde) for döngülerinden yararlanılmıştır.
3. String Modülü ve Metin İşleme: Kullanıcıların veritabanında arama yaparken büyük/küçük harf duyarlılığından etkilenmemesi adına (lower, upper metodları) ve formlardan gelen verilerdeki gereksiz boşlukların temizlenmesi (strip metodu) için string işlemleri aktif olarak kullanılmıştır.
4. Fonksiyonlar (Functions): Spagetti kod yazımını engellemek, kodun okunabilirliğini ve modülerliğini artırmak için CRUD işlemleri (Ekleme, Okuma, Güncelleme, Silme) database.py içerisinde parametre alan ve değer döndüren bağımsız fonksiyonlar olarak tasarlanmıştır.
5. Listeler (Lists): Veri tabanından fetchall() metoduyla dönen sorgu sonuçları (satırlar), Python içinde listeler halinde tutulmuş, listelerin append() ve extend() gibi fonksiyonları ile veri manipülasyonu yapılmıştır.
6. Demetler (Tuples): SQLite execute() komutlarına dışarıdan parametre gönderilirken, SQL Injection (veritabanı hackleme) saldırılarını engellemek ve parametre yapısını sabit tutmak amacıyla veriler tuple formatında iletilmiştir.
7. Sözlükler (Dictionaries): Ana sayfada (Dashboard) gösterilen özet veriler (toplam kategori, toplam parça, toplam maliyet vb.) tek bir sözlük yapısı (dict) içinde toplanmış, anahtar-değer (key-value) ilişkisiyle arayüze yönlendirilmiştir.
8. Özyinelemeli (Recursive) Fonksiyonlar: Projedeki kategori sistemi, sınırsız derinliğe (Örn: Motor > Pistonlar > Alt Pistonlar) sahip olabilecek şekilde tasarlandığı için, kendi içindeki alt kategorileri bulana kadar kendini tekrar çağıran özyinelemeli fonksiyonlar (kategori_agaci) kullanılmıştır.
9. Modüller (Modules): Yazılımın farklı yetenekler kazanması için Python'ın yerleşik ve harici modülleri import edilmiştir (os, csv, io, datetime, sqlite3, flask).
10. Dosya İşlemleri (File I/O): Sistemdeki hataların ve kritik işlemlerin takibi için "with open()" metodu kullanılarak uygulama.log adında bir metin dosyası oluşturulmuş ve append moduyla loglama yapılmıştır. Ayrıca veritabanındaki verilerin CSV (Excel) dosyası olarak dışa aktarımı sağlanmıştır.

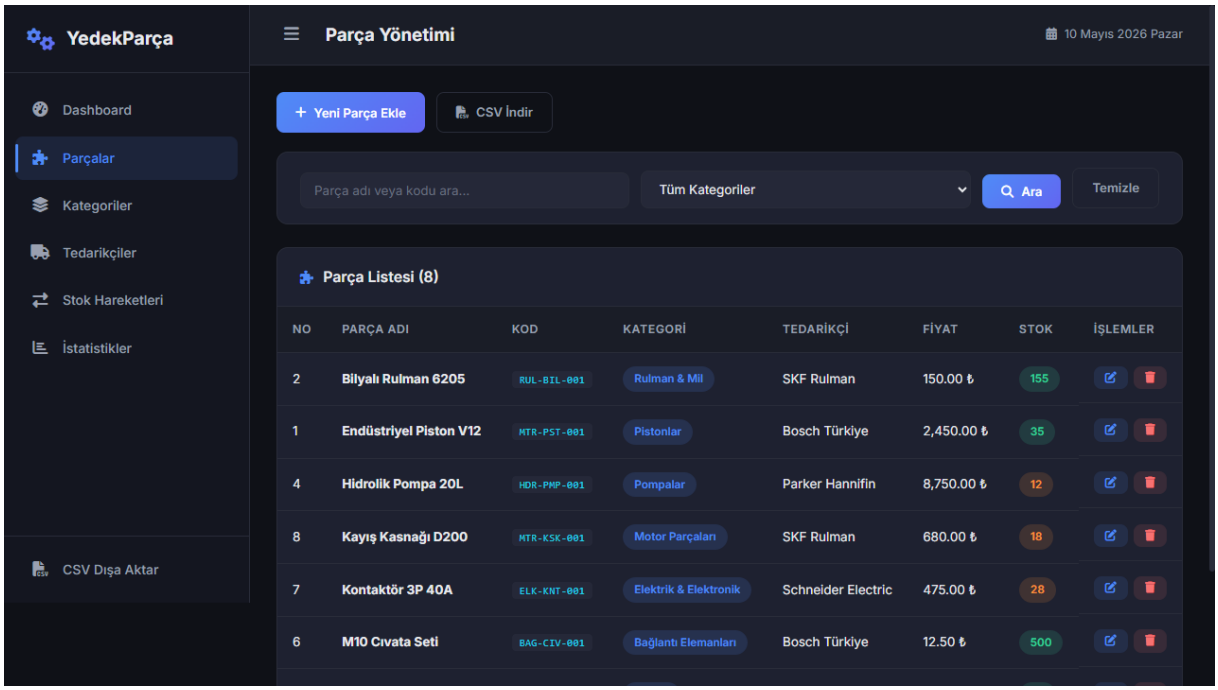
11. İstisna Yönetimi (Try/Except/Finally): Kritik kod bloklarında (özellikle veritabanına veri yazma/silme anında) programın hata verip çökmesini önlemek için hata yakalama blokları kullanılmış, bağlantıların her halükarda kapanması için finally blokları eklenmiştir.
12. Nesne Yönelimli Programlama (OOP): Kodun daha profesyonel bir standartta olması için models.py dosyasında class'lar oluşturulmuş, veritabanı satırları sınıf nesnelerine dönüştürülmüştür.
13. Numpy / Pandas / Matplotlib: Verilerin sadece listelenmesi yeterli görülmeyip; Pandas ile DataFrame'e dönüştürülmüş, Numpy ile istatistiksel hesaplamaları yapılmış ve Matplotlib kullanılarak web sayfasına doğrudan yansıyan PNG formatında grafikler oluşturulmuştur.

4. UYGULAMA ARAYÜZÜ VE EKRAN GÖRÜNTÜLERİ

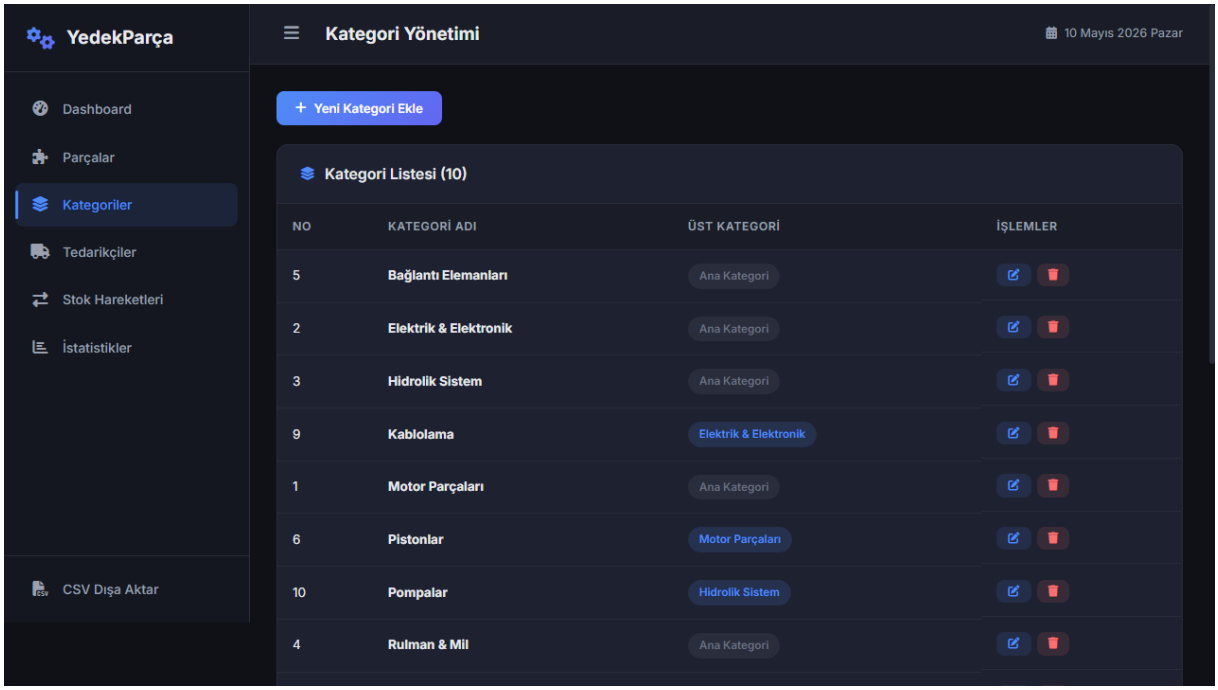
Sistemin ön yüzü (front-end), kullanıcı dostu ve sezgisel bir deneyim sunması amacıyla modern web tasarım prensipleri gözetilerek geliştirilmiştir. Tüm sayfalar mobil uyumlu (responsive) bir yapıdadır. Projede, kullanıcıya başarılı işlemler veya hatalar hakkında anında bilgi verebilmek için (örneğin: "Parça başarıyla güncellendi" veya "Bu e-posta zaten kayıtlı") dinamik toast (bildirim) mesajları kullanılmıştır. Tasarım aşamasında CSS kodlamaları için yapay zeka destekli araçlardan genel rehberlik alınmış ve proje ihtiyaçlarına göre özelleştirilmiştir.



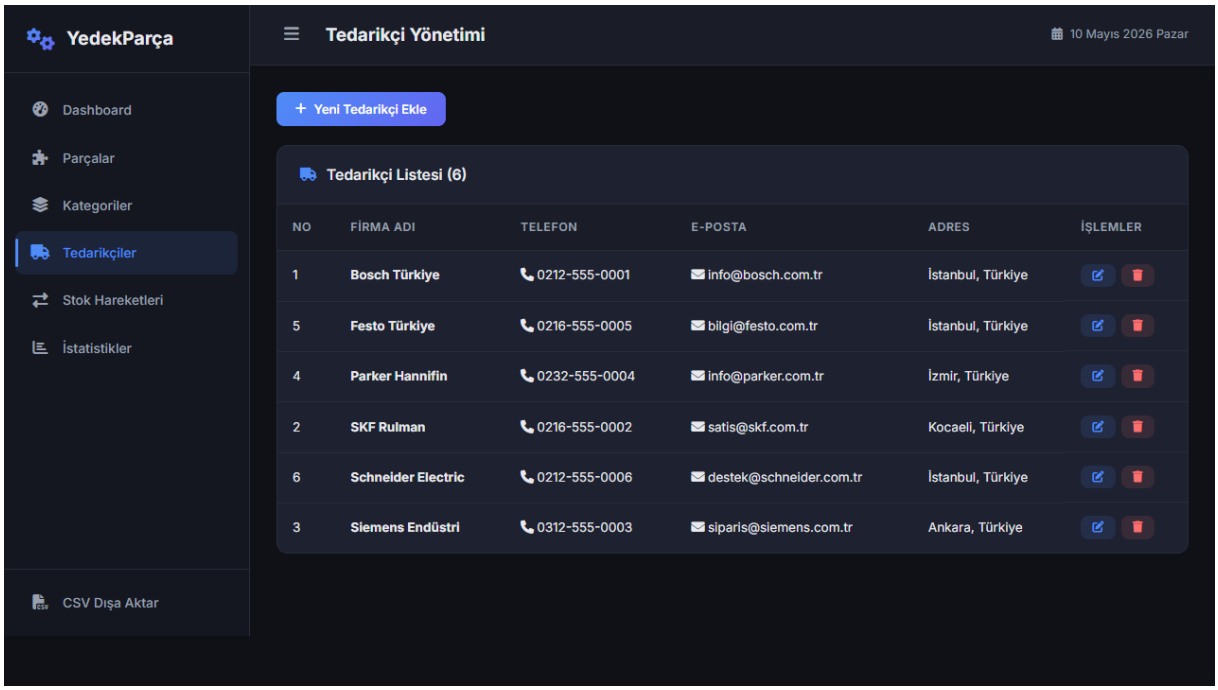
Şekil 1: Sistemin Ana Sayfası (Dashboard) ve Özet Stok Durumu Ekranı



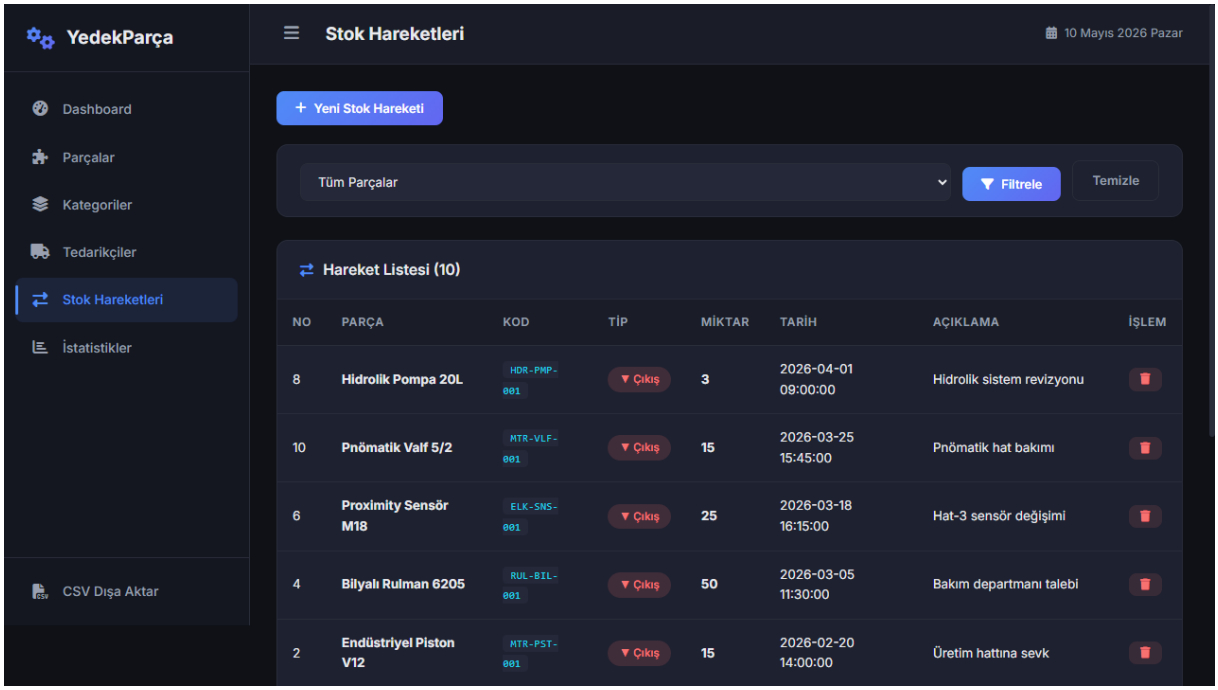
Şekil 2: Yedek Parçaların Listelendiği ve Yeni Parça Ekleme İşlemlerinin Yapıldığı Arayüz



Şekil 3: Özyinelemeli (Recursive) Fonksiyonla Oluşturulan Kategori Yönetim Sayfası



Şekil 4: Tedarikçi Bilgilerinin Tutulduğu ve İletişim Detaylarının Yönetildiği Ekran



Şekil 5: Parçalara Ait Giriş/Çıkış Hareketlerinin ve Logların Görüntülendiği Sayfa



Şekil 6: Numpy, Pandas ve Matplotlib Kütüphaneleri ile Çizdirilen Veri Analizi ve İstatistik Grafikleri

5. KARŞILAŞILAN ZORLUKLAR VE ÇÖZÜM YAKLAŞIMLARI

Böylesine kapsamlı bir sistem sıfırdan geliştirilirken çeşitli yazılımsal ve mantıksal mimari zorluklarıyla karşılaşılmıştır:

1. Kategori Hiyerarşisini Dinamik Oluşturma Sorunu:

Zorluk: Klasik ilişkisel veritabanlarında tablolar genellikle yatay bağlanır. Ancak bu projede, bir kategorinin alt kategorisi, onun da alt kategorisi (sınırsız derinlik) olması gerekiyordu. Standart SQL JOIN sorguları bu ağaç yapısını dökmekte yetersiz kalıyordu.

Çözüm: Bu sorunu aşmak için Python'un algoritmik gücünden faydalanılmış ve veri tabanını tarayarak alt kırılımları bulan "Özyinelemeli (Recursive) Fonksiyon" yazılmıştır. Bu sayede hiyerarşi istenildiği kadar derinleşebilmektedir.

2. Web Üzerinden Matplotlib Grafiği Sunma Sorunu:

Zorluk: Matplotlib kütüphanesi yapı gereği masaüstü uygulamalarında (GUI) yeni bir pencere açarak grafikleri gösterir. Flask tabanlı bir web uygulamasında grafiği çizdirdiğimizde sunucu (backend) tarafında pencere açılmaya çalışıyor ve uygulama çöküyordu.

Çözüm: Matplotlib'in arayüzsüz çizim motoru olan "Agg" (Anti-Grain Geometry) backend'i devreye sokuldu. Çizilen grafikler sunucuda fiziksel bir resim dosyası (PNG) olarak kaydedilmek yerine, "io.BytesIO()" kullanılarak doğrudan RAM (bellek) üzerinde bayt dizisine çevrildi ve HTML sayfasına dinamik olarak fırlatıldı.

3. Kaskad Silme ve Veri Bütünlüğü (Orphan Data) Sorunu:

Zorluk: Sistemde bir tedarikçi veya kategori silindiğinde, o kategoriye ait yüzlerce yedek parça veritabanında "boşlukta" (yetim kayıt) kalacaktı. Bu da veri tabanını çöplüğe çevirebilirdi.

Çözüm: Veritabanı şeması (schema.sql) tasarlanırken Foreign Key (Yabancı Anahtar) ilişkilerine sıkı kısıtlamalar getirildi. İçinde parça bulunan bir kategori silinmek istendiğinde "ON DELETE RESTRICT" kısıtlaması ile sistemin hata fırlatması (ve try-except ile bu hatanın yakalanarak kullanıcıya "Önce içindeki parçaları silin" uyarısı verilmesi) sağlandı. Stok hareketlerinde ise parça silinirse logların da otomatik silinmesi için "ON DELETE CASCADE" yapısı kuruldu.

6. SONUÇ

Bu dönem sonu projesi bağlamında geliştirilen "Sanayi Yedek Parça Yönetim Sistemi", Programlamaya Giriş II dersi boyunca teorik düzlemde öğrenilen Python temellerinin, gerçek dünya problemlerini çözebilecek tam teşekküllü bir otomasyon yazılımına nasıl dönüştürülebileceğinin en somut kanıtıdır. Klasik ve durağan bir ödev kurgusunun ötesine geçilerek; kullanıcı dostu bir arayüze (UI), sağlam bir arka uç (back-end) mimarisine ve veri bütünlüğünü koruyan ilişkisel bir veri tabanı (SQLite) modeline sahip, yaşayan bir web uygulaması ortaya konmuştur.

Geliştirme süreci, yalnızca kod yazma pratiği kazandırmakla kalmamış; aynı zamanda algoritmik düşünme, hata yakalama (Exception Handling) ve sistem dizaynı konularında ciddi bir vizyon katmıştır. Proje, standart CRUD (Ekle, Oku, Güncelle, Sil) operasyonlarını yerine getirmenin çok daha ötesine geçerek; Pandas ile veri madenciliği yapabilen, Numpy ile istatistiksel hesaplamaları yürüten ve Matplotlib entegrasyonu sayesinde elde ettiği verileri yönetim kademesi için görsel grafiklere (Dashboard) dönüştürebilen analitik bir platform yeteneğine kavuşturulmuştur. Veritabanı tasarımında kullanılan kaskad (CASCADE) yapılar ve özyinelemeli (recursive) algoritmalar sayesinde uygulamanın mantıksal sınırları genişletilmiştir.

Sonuç olarak; geliştirilen bu sistem, sanayi veya depo sektöründeki bir işletmenin günlük yedek parça operasyonlarını dijitalleştirecek kapasitededir. Proje mimarisi modüler (genişlemeye açık) bir yaklaşımla inşa edildiği için; ilerleyen süreçlerde barkod okuyucu entegrasyonu, yetkilendirmeli personel girişi (Login/Role sistemi) ve fatura/irsaliye yazdırma gibi eklentilerle çok daha büyük bir ERP (Kurumsal Kaynak Planlaması) yazılımına dönüştürülmeye tamamen hazırdır. Bu çalışma, modern yazılım geliştirme süreçlerinin tüm katmanlarını (Full-Stack) deneyimlemek adına paha biçilemez bir akademik tecrübe olmuştur.

7. KAYNAKÇA

1. Python Software Foundation, "Python 3.10 Documentation", https://docs.python.org/3/
2. Pallets Projects, "Flask Web Development Framework Documentation", https://flask.palletsprojects.com/
3. McKinney, W., "Data Structures for Statistical Computing in Python", https://pandas.pydata.org/docs/
4. Hunter, J. D., "Matplotlib: A 2D graphics environment", https://matplotlib.org/stable/gallery
5. Ön yüz (Front-end) tasarımında kullanılan CSS stilleri ve arayüz yerleşimi için yapay zeka asistanlarından (ChatGPT / Claude) fikir ve kod bazlı tasarım rehberliği alınmıştır.
6. DB Browser for SQLite Resmi Belgeleri, https://sqlitebrowser.org/